DarnerView: Journal of Artificial Intelligence

Year 2026 • Manuscript submitted for review
e-ISSN: assigned by the journal
https://www.darnerview.com

DOI: to be assigned by the journal

RESEARCH ARTICLE

Selective Invalidation of Procedural Agent Memory under Tool Contract Changes in a Controlled Simulation

Moslem Mohseni Khah[1*], **Sepide Ebrahimian Maqsoud**[2], **Navid Sadeghi**[3]

[1] Department of Computer Engineering, Islamic Azad University, Hamedan Branch, Hamedan, Iran
[2] Department of Electrical Engineering, Hamedan University of Technology, Hamedan, Iran
[3] Department of Electrical Engineering – Control, Shahid Beheshti University, Tehran, Iran

**Corresponding author: Moslem Mohseni Khah • Email: moslem.mohsenikhah@iau.ir*

Received: 26.10.2026 • Accepted: — • Published: —

ABSTRACT

Procedural memories can become stale when tool interfaces change, yet clearing a complete skill library discards valid experience. This simulation study evaluates maintenance decisions immediately after a known contract change. An executable environment distinguishes visible argument or precondition errors from silent unit errors and harmless optional additions. Eight policies are compared on 300 paired random worlds per condition, with controlled dependency-recording recall, probe sensitivity and relearning accuracy. A parametric agent represents execution and repair; its probabilities are assumptions rather than measurements of a language model. Tool-level selection achieved success 0.894 versus 0.894 for exhaustive revalidation, while reducing probe effort by 38.5%. Clause-level selection changed success by −0.97 percentage points relative to exhaustive revalidation. Fresh-seed confirmation, a larger library and a simple trace extractor examine robustness and the operational meaning of these trade-offs. No policy dominates across assumptions: selective probing reduces maintenance effort, incomplete dependency records leave silent failures, and clearing can improve success when probes are unreliable. The results characterise a controlled maintenance problem and require validation with real model-driven agents.

Keywords: procedural memory; tool contracts; selective invalidation; agent reliability; dependency recording; simulation

© 2026 The Author(s). This article is licensed under a Creative Commons Attribution-NonCommercial 4.0 International License (CC BY-NC 4.0): https://creativecommons.org/licenses/by-nc/4.0/

1. Introduction

An agent that retains an executable procedure must decide whether yesterday's procedure remains valid today. Skill reuse can reduce repeated planning, as illustrated by Voyager, while reflective memory can preserve lessons from failed attempts (G. Wang et al., 2023; Shinn et al., 2023). Neither representation makes an old tool assumption permanently valid. Renaming an argument produces an obvious error; changing its unit can instead produce a plausible response with the wrong effect. Interface evolution is a documented software-maintenance problem (Brito et al., 2018).

At a known change event, keeping every skill preserves both valid and stale procedures. Clearing every skill removes staleness but forces relearning. Revalidating every skill spends work on unaffected procedures. Selective maintenance records dependencies between skills and contract clauses, then probes only records touched by the diff. Its value depends on whether those dependencies are complete and whether a probe detects the relevant error.

Dependency-directed revision has a long history in truth maintenance (Doyle, 1979). Recent memory systems also retain verification metadata or related memory units (H. Wang et al., 2026;

Yang et al., 2026). Selective skill retention itself has already been tested by R. Wang et al. (2026). The contribution here is a controlled measurement at an externally specified tool-contract change, rather than a new general principle of selective memory.

The study asks: RQ1, how do selective policies compare with retention, expiry, clearing and exhaustive revalidation in success and cost? RQ2, how do dependency granularity and completeness affect that comparison? RQ3, which error, detection, relearning and scale assumptions change the practical choice?

2. Literature Review

2.1. Procedural reuse and memory governance

Voyager stores executable skills for an embodied environment; Reflexion stores verbal feedback to guide later attempts (G. Wang et al., 2023; Shinn et al., 2023). These establish distinct reusable objects, not a universal maintenance policy. MemGuard persists verifier-derived descriptors for later retrieval and retirement (H. Wang et al., 2026). Its full text discusses stale interface hints and dependence on imperfect verification. Our probe-sensitivity parameter isolates that dependence without reproducing its memory induction or language-model verifier.

MACE preserves conditions, actions and outputs in linked functional memory units (Yang et al., 2026). Its dependency-retention and noisy-retrieval experiments overlap with our concern for incomplete metadata, but its graph composition and task-feedback loop differ from a single known contract diff. Its imported auxiliary baselines differ from its locally controlled comparisons; their headline scores are not treated as comparable evidence here.

Anti-Persistence reconstructs temporary guidance from trajectories, while its selective-retention appendix tests filtering and a frozen skill layer (R. Wang et al., 2026). That appendix precludes a novelty claim for selective persistence alone. Our experiment measures invalidation coverage and maintenance cost under executable external changes; it does not model iterative abstraction descendants or reproduce error reproduction.

2.2. Executable evaluation and the remaining question

Function-calling evaluation distinguishes syntactic invocation from stateful reasoning (Patil et al., 2025). Tau-bench evaluates final database state rather than relying on the agent's success claim (Yao et al., 2024). Ambiguous tool outcomes likewise distinguish model behaviour from harness and contract effects (Li, 2026). We use executable postconditions for the same methodological reason: a successful-looking return is insufficient for task success. AgentMemoryBench is a possible future transfer environment, not a benchmark run in this study (solomoon313, n.d.).

The unresolved experimental question is how much maintenance work can be avoided while preserving executable validity, given incomplete dependency metadata. Our setting makes the diff known and the task evaluator exact. It leaves contract discovery, semantic parsing of natural-language changes and open-ended planning outside the evidence. This narrow boundary permits attribution of policy differences to explicit assumptions.

3. Materials and Methods

3.1. Executable environment and paired design

Study 1 generates 300 worlds per condition. Each contains 40 task types and 12 tools; a skill uses three to six tools. The warm-started library is evaluated for 400 task steps, with one change at step 100 and 300 post-change tasks. A Zipf task-frequency distribution gives repeated exposure to popular skills. All policies receive the same world, task stream and execution draws. Policy-specific maintenance consumes a separate deterministic random stream; pairing is by world rather than by individual probe outcome.

A contract declares accepted argument names, unit factors and prerequisite tools. A stored skill records the names and factors used when learned. Execution checks argument acceptance and prerequisite order, then compares interpreted values with intended values. Missing requested outputs are failures. Renames and unmet prerequisites are visible errors; changed units can silently alter final state. Optional additions are harmless by construction. The default diff touches 2 tools, with equally weighted change types. Requested prerequisite changes on the lowest-index tool fall back to a rename; the prerequisite regime therefore includes that boundary case rather than a strictly pure intervention.

3.2. Dependency recording and maintenance policies

The main recorder independently retains true clauses with probability 0.9. Tool-level selection flags a skill if any recorded clause refers to a changed tool; clause-level selection requires a clause intersection. This is a controlled missingness model, not a measured natural-language extractor. A separate exploratory extractor reads structured call traces, recording argument, unit and prerequisite-contract reads. An arguments-only ablation omits prerequisite reads. Recall is measured against the executable grammar's dependency set; it makes no claim about arbitrary program dependencies.

- Keep all retains the library; clear all deletes it; time expiry removes skills older than the configured window.
- Revalidate all probes every skill; selective tool and selective clause probe only flagged records.
- Clause with random audit also probes a random tenth of unflagged records; clause with tiered probe samples half of the records sharing a changed tool but lacking a changed clause.

The expiry window is 75 steps. Probes always detect visible errors and detect silent errors with probability 0.7; each costs 0.5 call-equivalents. A failed probe deletes the skill for later relearning. The tiered strategy was introduced after exploratory inspection, so its generality is assessed on a separate confirmation stream. No policy receives hidden task-success labels during task execution.

3.3. Agent assumptions and outcome accounting

A valid stored skill succeeds with probability 0.95. Missing memory requires exploration, with success probability 0.75 and a cost of twice the requested tool count plus two calls, excluding any added prerequisite tools. Visible failure can trigger repair, costing 3 calls with success probability 0.85. Silent failure is not noticed in the main condition. These are assumptions, not calibrated

language-model measurements. The simulator records exact outcome labels and separately counts visible and silent stale executions.

The primary outcomes are post-change success and mean call-equivalent cost. Cost equals execution, exploration, repair and maintenance cost divided by post-change tasks. Secondary outcomes are probe effort, stale executions, stale skills remaining after maintenance and healthy retention. Because cheap silent failure can look efficient, harm-weighted cost separately charges each silent failure; this is a preference sensitivity, not an estimated monetary loss. Actual tokens and latency are not represented by call-equivalents.

3.4. Statistical analysis and exploratory separation

Main seeds are zero through 299; fresh confirmation seeds start at 10000. Added scale and extractor conditions use seeds starting at 20000. Percentile bootstrap intervals use 2000 resamples across worlds (Efron, 1979). Paired mean differences use two-sided sign-flip tests with 3000 permutations. Holm correction covers the complete reported family of paired success, cost and probe comparisons (Holm, 1979), conservatively including the secondary probe outcome. Intervals are nominal rather than simultaneous. The family was inherited from exploratory development and is not a preregistration; confirmation and new sensitivities are reported separately, with no tuning on their outcomes.

3.5. Study 2 protocol and ethics

A separate model-driven harness replaces parametric execution with a language-model-generated plan on a held-out world stream, comparing retention, clearing, exhaustive revalidation and both selective granularities. Prompts expose the current contract and available stored plan; exact executable effects judge success, including omitted or invalid steps. Model calls and maintenance probes remain separate because their units differ. Responses are cached by prompt, model, seed and decoding configuration; paid calls require conservative pricing and a persistent per-paper budget cap. Mock responses test only software plumbing and are excluded from scientific results. Only synthetic tasks are used, without human participants or personal data.

4. Results

4.1. Success and maintenance cost

Table 1. *Main simulation comparison*

Policy	Success and interval	Cost and interval	Probe cost	Silent failures
Keep all	0.798 [0.775, 0.820]	1.18 [1.17, 1.20]	0.0	45.7
Clear all	0.918 [0.917, 0.920]	2.60 [2.58, 2.61]	0.0	0.0
Time expiry	0.873 [0.870, 0.876]	4.19 [4.17, 4.22]	0.0	3.7
Revalidate all	0.894 [0.884, 0.903]	1.74 [1.71, 1.77]	20.0	13.5
Selective tool	0.894 [0.884, 0.903]	1.72 [1.68, 1.75]	12.3	13.5
Selective clause	0.884 [0.873, 0.895]	1.66 [1.63, 1.70]	9.0	16.7
Clause with random audit	0.885 [0.873, 0.895]	1.67 [1.64, 1.71]	10.1	16.5
Clause with tiered probe	0.893 [0.884, 0.902]	1.70 [1.66, 1.73]	10.7	13.9

Note. Nominal 95% bootstrap intervals across paired worlds. Probe cost is call-equivalent effort, not a probe count. Silent failures are counts per post-change world.

For RQ1, Table 1 shows success 0.918 for clearing and 0.798 for retention. Tool selection versus exhaustive revalidation had a paired success difference 0.0000 [0.0000, 0.0000], and cost difference −0.0256 [−0.0262, −0.0250]. A small estimated success difference is not proof of equivalence; no equivalence margin was specified.

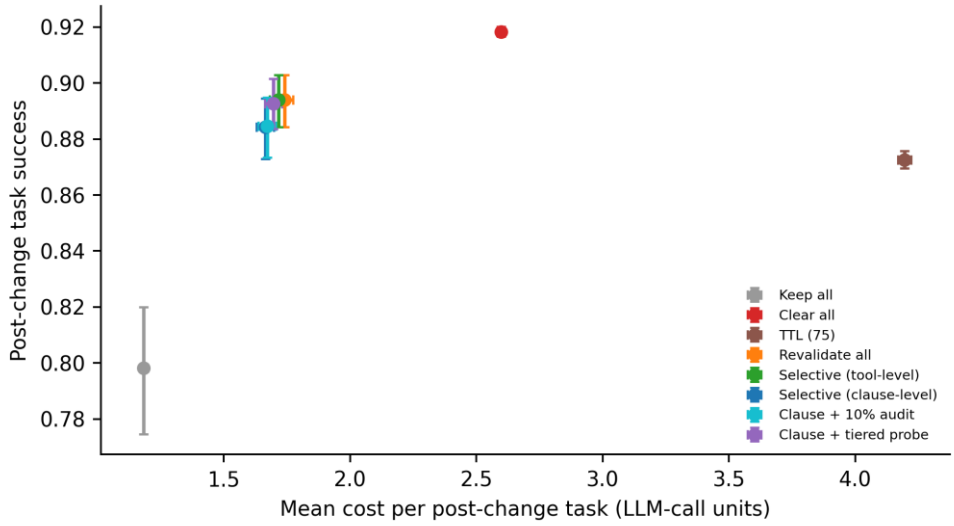


Figure 1. *Success and call-equivalent cost in the main condition.*

4.2. Paired effects and recorded-dependency quality

Table 2. *Selected paired mean differences and Holm-adjusted tests*

Comparison	Outcome	Difference and interval	Adjusted p
Selective tool vs Revalidate all	Success	0.0000 [0.0000, 0.0000]	1.0000
Selective tool vs Revalidate all	Cost	−0.0256 [−0.0262, −0.0250]	0.0080
Selective clause vs Revalidate all	Success	−0.0097 [−0.0132, −0.0066]	0.0080
Selective tool vs Clear all	Success	−0.0244 [−0.0340, −0.0154]	0.0080
Selective tool vs Clear all	Cost	−0.8809 [−0.9149, −0.8472]	0.0080

Note. First policy minus second; tests belong to the complete family supplied in the supplementary results, not only these displayed comparisons.

For RQ2, clause-level success was 0.894 with complete records and 0.847 with half the dependencies recorded. Corresponding tool-level success was 0.894 and 0.891. Redundant clauses referring to a changed tool help flag records even when individual clauses are missing. This robustness depends on simulated redundancy and should not be extrapolated to sparse real traces.

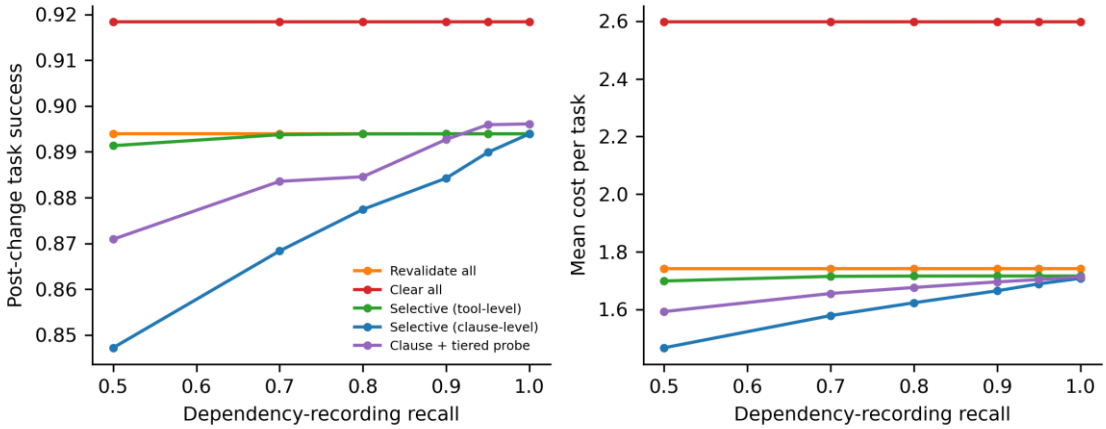


Figure 2. *Dependency completeness changes the success and cost trade-off.*

4.3. Error type and assumption sensitivity

For RQ3, pure silent unit changes produced success 0.357 for retention, 0.766 for exhaustive revalidation and 0.918 for clearing. For harmless changes, retention achieved 0.950 and clearing 0.918. A benign diff alone does not justify deletion; maintenance value depends on silent errors and unnecessary relearning.

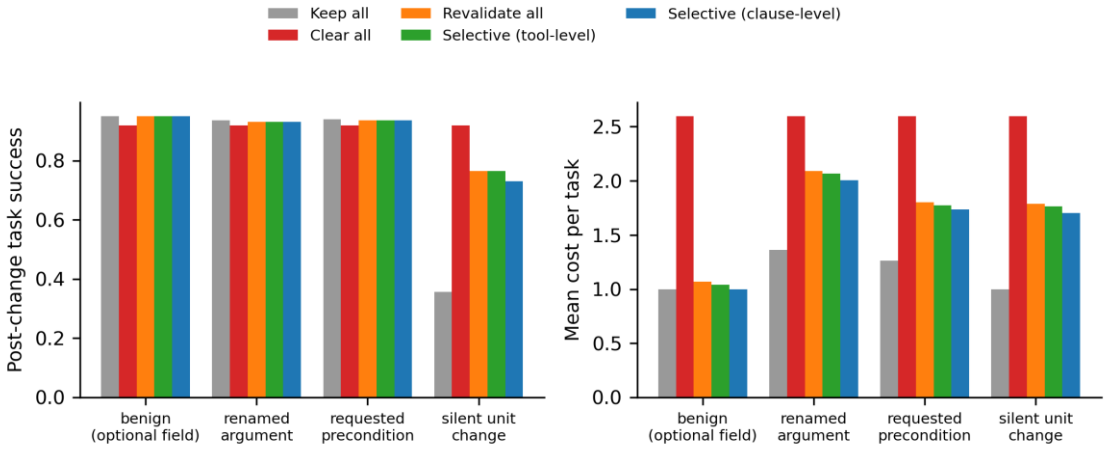


Figure 3. *Success and cost by executable contract-change type.*

With perfect silent-error detection, tool-level success was 0.936; with weak detection it was 0.844. At exploration accuracy 0.5, clearing achieved 0.851 versus 0.867 for selection. These reversals make a universal ranking inappropriate. Probe-cost and feedback sweeps are supplied so alternative preferences can be examined without selecting a favourable weight after inspection.

4.4. Confirmation and exploratory scale and trace tests

On fresh confirmation worlds, success was 0.919 for clearing, 0.892 for exhaustive probing, 0.892 for tool selection and 0.893 for the exploratory tiered strategy. This confirms only the specified conditions and does not convert exploratory policy design into preregistered evidence.

The added library contains 100 skills and 30 tools. Selection achieved 0.922 success at cost 1.59; exhaustive revalidation achieved 0.922 at cost 1.71. The horizon is unchanged, so this jointly changes size and utilisation rather than isolating an asymptotic scaling effect.

The full-trace extractor had measured clause recall 1.000; omitting prerequisite reads reduced it to 0.777. Clause-level success was 0.890 and 0.891, respectively. Omitting prerequisite reads need not reduce success here because visible repair and relearning have different assumed accuracies. Exact grammar knowledge favours the complete extractor; its recall is not an estimate for natural-language or opaque external tools.

4.5. Study 2 execution status

No real language-model result is reported. Provider discovery found no API credentials or ready local model. The local-runtime acquisition attempt did not provide an executable model during this study. The held-out harness completed its mock dry run, which checks parsing, caching and exact scoring but provides no estimate of model success. No confidence interval or model comparison is inferred from mock output.

5. Discussion

Selective maintenance buys avoided probes; it does not guarantee a better task outcome. A sound decision must consider both affected-skill coverage and the probe's ability to detect semantic errors. Tool-level redundancy can rescue missing clauses in this grammar, while clause precision saves work by refusing to examine harmless overlaps. Its benefit and risk arise from the same filter. A lower cost can conceal undetected incorrect executions rather than reflect a more effective agent.

Clearing is a strong comparator because it removes silent staleness before any probe is trusted. It also discards valid skills and pays exploration costs. Whether this is worthwhile depends on library utilisation, recovery accuracy and the cost of a wrong effect. The larger-library sensitivity changes utilisation together with size; it cannot establish a general scaling law. An implementation should estimate its own workload and verifier errors before treating this simulation as a deployment decision.

The results complement memory governance. MemGuard motivates retaining verification evidence; MACE preserves dependencies; Anti-Persistence shows that persistent abstractions and filters behave differently across contexts (H. Wang et al., 2026; Yang et al., 2026; R. Wang et al., 2026). Our imposed diff gives exact attribution of stale records but removes the difficult task of identifying whether an observed failure reflects a changed contract. No experiment here ranks those complete systems.

Study 2 is a transfer test. Without real completions, the results support only the specified parametric regimes. A next test should freeze prompts and executable tasks, report invalid output and repair costs, and keep inference, probes and wall-clock latency separate. Model-driven evidence could reverse the policy ranking and would require revising the practical conclusions.

6. Conclusion

RQ1 is answered by a conditional success-cost trade-off: selective probing reduces maintenance effort without uniformly improving success. RQ2 is answered by dependence on metadata completeness and redundant tool-level coverage. RQ3 is answered by detection, exploration, change-type and scale sensitivities: the preferred policy changes with those assumptions. No

policy dominates across the studied regimes. The contribution is a reproducible comparison at an external contract-change event, with the limits of that comparison made explicit.

6.1. Recommendations

Record tool use and explicit contract reads before attempting fine-grained invalidation. Measure recorder omissions and semantic-probe sensitivity on known changes. When a clause filter is incomplete, examine tool-sharing records or require independent postconditions. Compare maintenance savings with relearning and wrong-effect costs under the actual workload, retaining clearing and exhaustive revalidation as baselines.

6.2. Limitations

The environment has one fully known change, fixed semantics and a warm-started library. Parametric execution, exploration and repair are uncalibrated; call-equivalents account for resources rather than measured inference cost. Independent missing clauses and exact structured traces omit correlated semantic misunderstandings. The tiered strategy followed exploratory inspection; fresh seeds limit reuse of that outcome stream but cannot remove researcher choices. Added scale and trace tests are exploratory, and the unchanged horizon confounds size with utilisation. No real model validation was completed, so planning, prompt sensitivity and language-mediated errors remain unmeasured. Repeated changes, shared subplans and production data may yield different trade-offs. Bootstrap uncertainty concerns simulation worlds and does not establish deployment frequencies.

Declarations

Ethics Committee Approval: Not applicable: synthetic tasks and simulated data; no human participants or personal data were involved.

Author Contributions: The authors take collective responsibility for this manuscript.

Conflict of Interest: The authors declare no conflict of interest.

Funding: This research received no external funding.

Data and Code Availability: Code, data and generated results: https://github.com/moslem-mohseni/selective-agent-memory-invalidation. Supplementary material: supplementary_p1.zip; also available from the corresponding author.

References

Brito, A., Xavier, L., Hora, A., & Valente, M. T. (2018). Why and how Java developers break APIs. *arXiv*. https://arxiv.org/abs/1801.05198

Doyle, J. (1979). A truth maintenance system. *Artificial Intelligence*, *12*(3), 231–272. https://doi.org/10.1016/0004-3702(79)90008-0

Efron, B. (1979). Bootstrap methods: Another look at the jackknife. *The Annals of Statistics*, *7*(1), 1–26. https://doi.org/10.1214/aos/1176344552

Holm, S. (1979). A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, *6*(2), 65–70. https://www.jstor.org/stable/4615733

Li, J. (2026). Where does exactly-once live? Model, harness, and tool-contract effects on duplicate side effects in LLM agents. *arXiv*. https://arxiv.org/abs/2609.29095

Patil, S. G., Mao, H., Yan, F., Ji, C. C.-J., Suresh, V., Stoica, I., & Gonzalez, J. E. (2025). The Berkeley Function Calling Leaderboard (BFCL): From tool use to agentic evaluation of large language models. In

Proceedings of the 42nd International Conference on Machine Learning (Vol. 267, pp. 48371–48392). PMLR. https://proceedings.mlr.press/v267/patil25a.html

Shinn, N., Cassano, F., Berman, E., Gopinath, A., Narasimhan, K., & Yao, S. (2023). Reflexion: Language agents with verbal reinforcement learning. *arXiv*. https://arxiv.org/abs/2303.11366

solomoon313. (n.d.). *AgentMemoryBench* [Computer software]. GitHub. https://github.com/solomoon313/AgentMemoryBench

Wang, G., Xie, Y., Jiang, Y., Mandlekar, A., Xiao, C., Zhu, Y., Fan, L., & Anandkumar, A. (2023). Voyager: An open-ended embodied agent with large language models. *arXiv*. https://arxiv.org/abs/2305.16291

Wang, H., Dong, G., Liang, H., Zhang, Z., Luo, J., Liu, C., Xue, C., & Tang, H. (2026). MemGuard: Persisting verifier signals for LLM-agent memory governance. *arXiv*. https://arxiv.org/abs/2608.21867

Wang, R., Panaganti, K., & Zhou, D. (2026). More memory, worse agents: Error reproduction and anti-persistence in LLM agents. *arXiv*. https://arxiv.org/abs/2610.27406

Yang, K., An, M., Li, X., Yi, Z., Chen, Z., He, G., & Li, R.-H. (2026). MACE: Memory-agent co-evolution with adaptive memory graphs for multi-agent systems. *arXiv*. https://arxiv.org/abs/2609.21533

Yao, S., Shinn, N., Razavi, P., & Narasimhan, K. (2024). τ-bench: A benchmark for tool-agent-user interaction in real-world domains. *arXiv*. https://arxiv.org/abs/2406.12045